Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №64
ТЕМА: Створення сучасних інтерактивних міні-ігор або тренажерів на CustomTkinter.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- узагальнити навички побудови анімації, обробки подій та виявлення зіткнень у єдиному проєкті;
- ознайомитися з базовою архітектурою простої гри: стан, оновлення, відображення;
- навчитися реалізовувати підрахунок очок та умови завершення гри;
- отримати практичний досвід розробки завершеного інтерактивного застосунку розважального чи навчального призначення.

Теоретичні відомості

1. Архітектура простої гри: цикл «оновлення-відображення»

Будь-яка проста гра, незалежно від жанру, побудована на повторенні одного й того самого циклу дій, що виконується знову і знову (найчастіше — через after(), як у попередніх практичних роботах): 1) обробити введення користувача (натискання клавіш чи кліки); 2) оновити стан гри (позиції об'єктів, рахунок, перевірка зіткнень); 3) перемалювати або оновити відображення відповідно до нового стану. Розуміння цього циклу — ключ до проєктування будь-якої, навіть значно складнішої за навчальну, гри.

2. Стан гри як набір змінних

Стан гри — це сукупність усіх значень, які визначають поточну ситуацію: позиції об'єктів, рахунок, кількість життів, чи триває гра. У простих навчальних іграх стан зазвичай зберігають у звичайних змінних або невеликому словнику:

```
score = 0
lives = 3
game_running = True
```

Функції-обробники подій (натискання клавіш, завершення анімаційного кроку) читають і змінюють саме ці змінні стану — вони є «єдиним джерелом правди» про поточну ситуацію в грі, аналогічно до принципу, розглянутого раніше для багатовіконних застосунків.

3. Приклад гри «лови м'яч» (спрощений арканоїд)

Розглянемо структуру простої гри: платформа (керована стрілками користувача) повинна відбивати м'яч, що падає й рухається по полотну з відскоками від меж, а при промаху рахунок гри не збільшується:

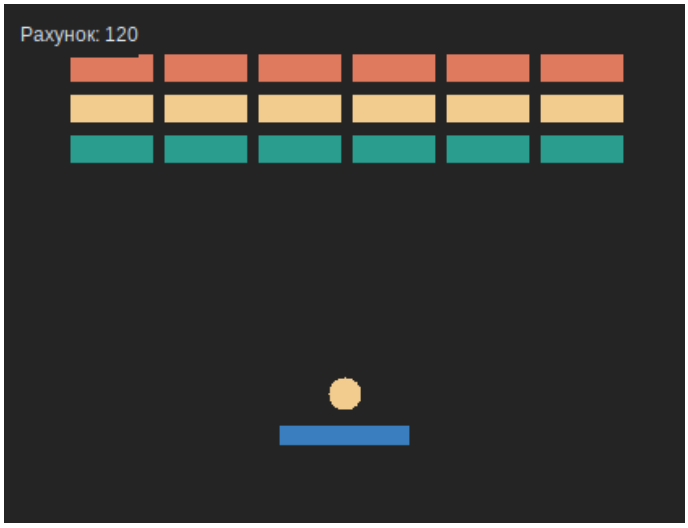


Рис. 1. Кадр із гри на кшталт «арканоїд»: цеглинки зверху, м'яч і платформа знизу, рахунок у верхньому куті

```
score = 0
ball_dx, ball_dy = 3, -3

def move_paddle(event):
    if event.keysym == 'Left':
        canvas.move(paddle, -20, 0)
    elif event.keysym == 'Right':
        canvas.move(paddle, 20, 0)

def game_loop():
    global ball_dx, ball_dy, score
    x1, y1, x2, y2 = canvas.coords(ball)
    if x1 <= 0 or x2 >= 380:
        ball_dx = -ball_dx
    if y1 <= 0:
        ball_dy = -ball_dy
    px1, py1, px2, py2 = canvas.coords(paddle)
    if is_colliding((x1, y1, x2, y2), (px1, py1, px2, py2)):
        ball_dy = -ball_dy
        score += 10
        score_label.configure(text=f'Рахунок: {score}')
    canvas.move(ball, ball_dx, ball_dy)
    app.after(30, game_loop)
```

Функція is_colliding() тут використовується так само, як у практичній роботі №63 — ця гра, по суті, є прямим практичним застосуванням усіх прийомів, вивчених у темі 6 (полотно, анімація, керування, зіткнення), об'єднаних в єдиний цілісний проєкт.

4. Умова завершення гри

Гра повинна мати чітку умову завершення — інакше вона триватиме нескінченно, навіть коли це вже не має сенсу (наприклад, м'яч «пролетів» повз платформу). Найпростіший спосіб реалізувати завершення гри — перевіряти умову програшу на кожному кроці циклу гри і, якщо вона виконується, зупиняти подальше планування after() (тобто просто не викликати game_loop() знову) та показувати відповідне повідомлення:

```
def game_loop():
    global game_running
    x1, y1, x2, y2 = canvas.coords(ball)
    if y2 >= 300:
        game_running = False
        canvas.create_text(190, 140, text='Гру завершено!',
fill='white', font=('Arial', 18, 'bold'))
        return
    # ... решта логіки руху й перевірок ...
    app.after(30, game_loop)
```

5. Поради щодо проєктування власної навчальної гри

Гра не обов'язково повинна бути розважальною в чистому вигляді — теми 6 добре підходить і для навчальних тренажерів (наприклад, тренажер таблиці множення, де правильна відповідь «влучає» у ціль, а неправильна — ні). Перш ніж писати код, варто чітко визначити: у чому мета гри для гравця; які об'єкти рухаються, а які статичні; яка подія означає успіх, а яка — невдачу; як рахуються очки.

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Тип гри та кількість життів залежать від N (mod — остача від ділення):

Завдання	Дані варіанта	Приклад N = 7
1	гра за N mod 3 = 0, 1, 2: «лови м'яч», «ухилення», «стрільба по цілях»	«ухилення»
4–5	кількість життів L = 3 + N mod 2	4

б) Кожне завдання — окремий файл Прізвище_ПР64_зN.py (наприклад, Ivanenko_PR64_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного завдання додайте скріншот вікна із видимим заголовком. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів і повідомлень — точно як в умові. Весь стан гри (очки, життя, позиції) тримайте в чітко названих змінних; кожен цикл after() запускайте лише один раз.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР64_з1.ру. Спроєктуйте гру за вашим варіантом і додайте до звіту короткий план (5–7 рядків): мета, керування, правила, очки, умова завершення.

   Приклад (N = 7): варіант «ухилення», життів 4.

2. Реалізуйте керований об'єкт, що рухається клавіатурою в межах полотна 400×400.

   Приклад: об'єкт не виходить за межі полотна.

3. Реалізуйте рухомі об'єкти (м'яч, перешкоди чи цілі) з автоматичним рухом і відскоками, де це доречно.

   Приклад: щонайменше 3 об'єкти рухаються одночасно.

4. Додайте CTkLabel «Очки: …» і «Життів: …», що оновлюються за подіями гри.

   Приклад: старт: «Очки: 0», «Життів: 4».

5. Реалізуйте завершення гри: коли життя закінчилися (або досягнуто мети), рух зупиняється й на полотні з'являється «Гру закінчено. Очки: …».

   Приклад (N = 7): після 4 зіткнень — «Гру закінчено. Очки: 12» (число залежить від гри).

6. Завдання підвищеної складності*. Додайте зростання складності (швидкість +1 кожні 50 очок або кожні 10 секунд) і кнопку «Почати заново», що скидає очки, життя й позиції об'єктів.

   Приклад: після «Почати заново» — «Очки: 0», «Життів: 4», усі об'єкти на початкових позиціях, один цикл after().

Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, стан гри розкиданий по коду, після завершення рух триває, «Почати заново» запускає другий цикл after(), немає очок чи життів) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити правило гри чи кількість життів). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Опишіть базовий цикл «оновлення-відображення», на якому побудована будь-яка проста гра.
2. Що таке стан гри і чому важливо тримати всі його складові в чітко визначених змінних?
3. Як реалізувати коректне завершення гри замість нескінченного продовження циклу анімації?
4. Наведіть приклад навчального (не розважального) застосування ігрової механіки з цієї теми.
5. Які кроки варто виконати на етапі планування перед написанням коду власної гри?